

VeriRuntime

固定目标的多验证器执行框架

从上层 DSL 到可追溯证据

设计与架构 / 一次真实执行 / 强证明的下一步

Docker Linux 本机复现

CBMC · ESBMC · CPAchecker
Ultimate · Frama-C · Rocq

22 页
约 20–25 分钟

上层固定目标，下层选择执行方式

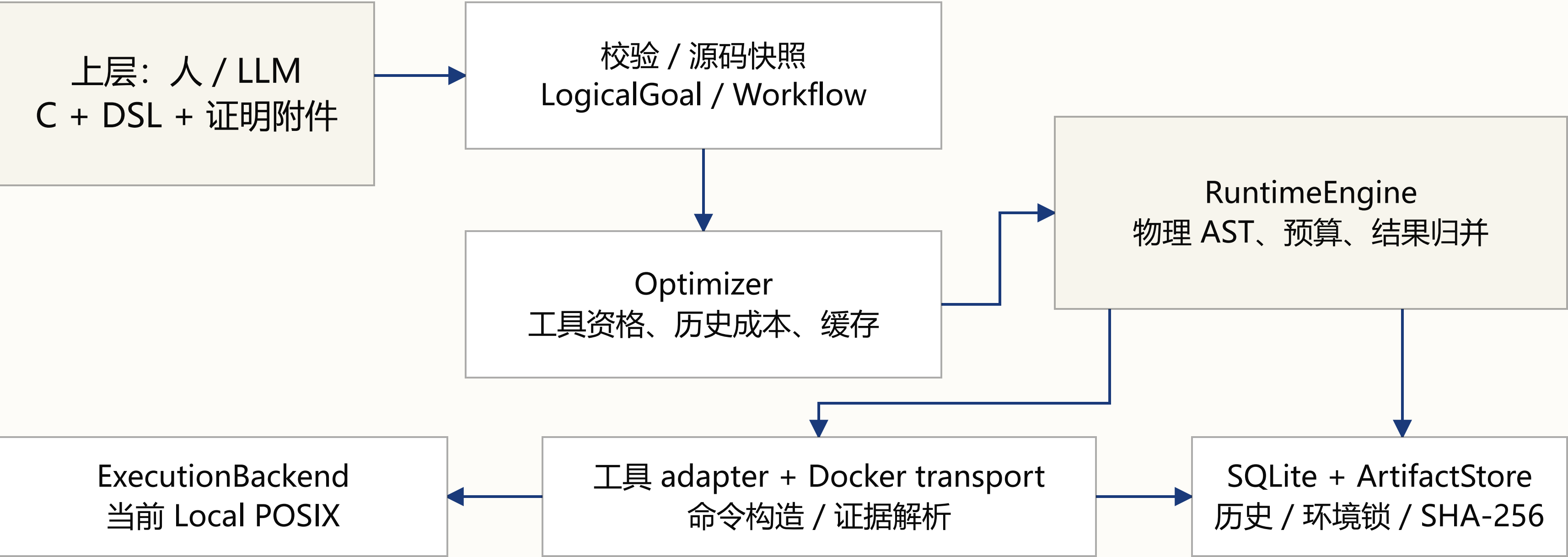
已有实现：验证执行、隔离、证据、复用；研究重点：效率与证据强度

角色	输入或职责	必须守住的边界
人 / 上层 LLM	源码、属性、语义、要求、预算 必要时编写 invariant / lemma / .v	只有上层能改变命题或假设
VeriRuntime	选择家族与顺序；并行 / 回退 核验缓存；解释与归并证据	优化执行方式，保持目标不变
ExecutionBackend	启动、等待、取消、指标、清理 执行已选择的 argv	返回原始执行事实，不判断 SAFE

当前验收主线：C 断言安全 + 至少两个不同家族确认

逻辑、调度与基础设施形成三层边界

上层目标进入调度；adapter 与 backend 分担命令解释、执行生命周期和原始证据



CLI → service → optimizer → engine → backend / adapter → evidence

一个语义目标，可以有多个物理计划

LogicalGoal 表示 WHAT; Run / Parallel / Sequence / CacheLookup 表示 HOW

对象	包含什么	什么时候变化
逻辑目标	源文件快照、入口、属性、C 语义 不同家族的最少确认数	上层明确提交语义变更
semantic_key	逻辑路径和内容 hash + 语义 + 要求	源码 / 命题 / 确认要求变化
物理 AST	具体工具、并行组合、时间片、回退 可包在 CacheLookup 中	工具环境 / 历史 / 预算变化
执行记录	goal execution id / attempt id 命令、日志、指标、结果、工件 hash	每次执行建立新记录

标签、预算、工具选择和 hints 不进入语义键；缓存还须独立校验环境与证据。

上层提供程序和要求，不指定验证器

例子：执行十次 `x++` 后，验证 `assert(x == 10)`

examples/c/safe_assert.c

```
#include <assert.h>

int main(void) {
    int x = 0;
    for (int i = 0; i < 10; ++i)
        x++;
    assert(x == 10);
    return 0;
}
```

examples/tasks/safe_assert_crosscheck.json

```
{
  "version": "0.1",
  "task": {
    "id": "safe-assert-crosscheck",
    "language": "C", "entry": "main",
    "sources": ["../c/safe_assert.c"]
  },
  "property": {"kind": "assertion_safety"},
  "semantics": {
    "c_standard": "c11", "data_model": "LP64"
  },
  "requirements": {"min_confirmations": 2},
  "budget": {
    "wall_time_sec": 30, "memory_mb": 2048,
    "max_parallel": 2
  }
}
```

输入只声明 WHAT：代码 + 属性 + 语义 + 证据要求 + 资源预算

JSON 变成不可漂移的 LogicalGoal

严格字段校验 → 解析 translation units / 本地头文件 → 保存内容快照

实际逻辑 IR 节选

```
id: safe-assert-crosscheck
entry: main
program.sources: ["c/safe_assert.c"]
program.files[0]:
  logical_path: c/safe_assert.c
  sha256: 48470bc7498b9918...
  content: "#include <assert.h>..."
property: assertion_safety
semantics: c11 / LP64
requirements.min_confirmations: 2
```

校验的主要边界

拒绝未知字段、工具名、策略与重复键
递归封存字面量本地 include
拒绝不支持的宏 / 绝对 / symlink 依赖
系统头文件依赖已资格验证的环境

semantic_key

c61c1d877544bae458d22cefd039eb13d4c38bcb6d01c549499050aab2591ad2

先筛兼容工具，再按历史成本排序

当前 cost-aware-v1 是启发式：平滑确定率 / 预期耗时

score = success_rate / expected_runtime

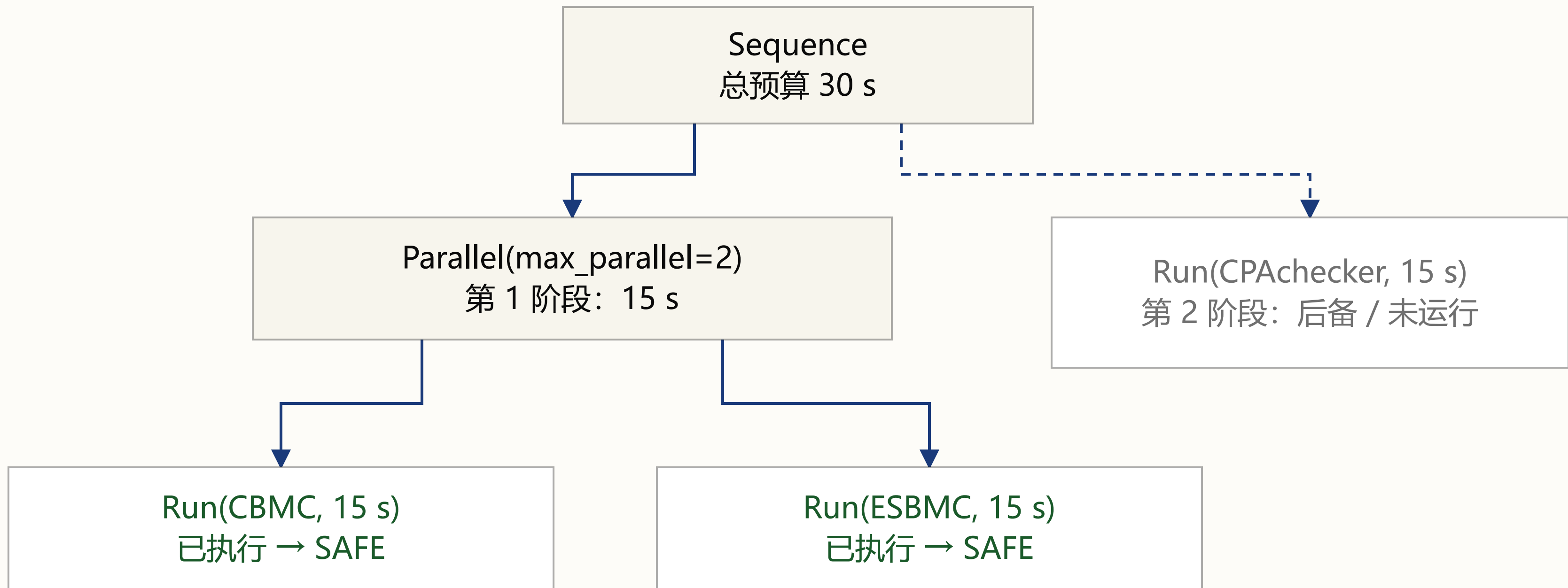
有历史：success = (definitive + 0.5) / (runs + 1); 耗时取历史中位数

首次无历史	平滑确定率	耗时先验 / 秒	score	选择
CBMC	0.5	0.5	1.000	第 1 阶段
ESBMC	0.5	0.7	0.714	第 1 阶段
CPAchecker	0.5	4.0	0.125	第 2 阶段 / 后备

可用性 / 语言 / 属性 / 语义 / 程序形态 / 内存估计 → 每家族最多一个候选

并行先确认，未满足才运行后备

Sequence[Parallel[CBMC, ESBMC], CPAchecker]



本例两家族 SAFE 已满足要求；运行时结束目标，跳过 CPAchecker。

执行后端接收命令，返回执行事实

当前接口已实现；远程 backend 的 staging / 环境资格验证仍留白

ExecutionSpec (真实 CBMC 桥接请求节选)

```
argv: ["python3.11", "-m",  
       "veriruntime.tools.container_driver",  
       "<当前 attempt 目录>"]  
cwd: <当前 attempt 目录>  
wall_time_limit: 14.864688 s  
cpu_request: 1.0  
memory_request_mb: 1024  
environment: private HOME / TMP / XDG  
metadata: execution / attempt / image identity
```

ExecutionOutcome (协议字段)

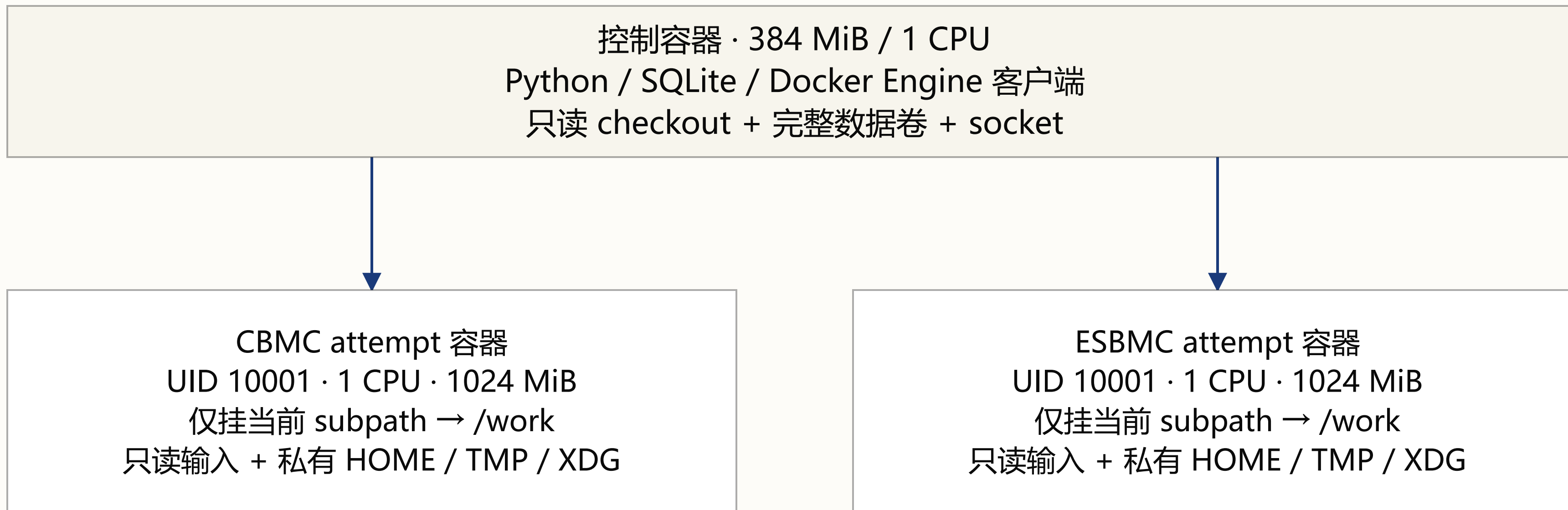
```
handle_id / backend  
exit_code / stdout / stderr  
start_time / end_time / wall_time_sec  
status / termination_reason  
metrics: RSS / CPU + scope  
diagnostics: resource_guarantees
```

start → wait / cancel → collect_metrics → cleanup

adapter 解析 Outcome；引擎归并 verdict。取消与清理必须幂等。

每次尝试一个容器，工作区之间不可见

轻量控制层 + 按需创建 / 删除的工具容器；没有常驻验证服务



共用限制：禁网 · 根目录只读 · cap_drop ALL · no-new-privileges · 128 PID · /tmp 64 MiB

工具读不到 peer / SQLite / checkout / socket；控制层持有 socket，属于可信本机 CLI。

两条实际命令，两份可解释的原始证据

命令由 adapter 派生；exit 0、属性状态与终端记录共同决定解释

CBMC 6.11.0 • native-command.json

```
/opt/cbmc/cbmc  
/work/input/c/safe_assert.c  
--function main --c11 --64  
--unwind 64 --unwinding-assertions  
--json-ui
```

ESBMC 8.5.0 • native-command.json

```
/opt/esbmc/release/bin/esbmc  
/work/input/c/safe_assert.c  
--function main --std c11 --64  
--unwind 64 --z3 --multi-property  
--overflow-check --ub-shift-check
```

CBMC stdout JSON 节选

```
"property": "main.assertion.1"  
"description": "assertion x == 10"  
"status": "SUCCESS"  
...  
"cProverStatus": "success"
```

ESBMC stderr 节选

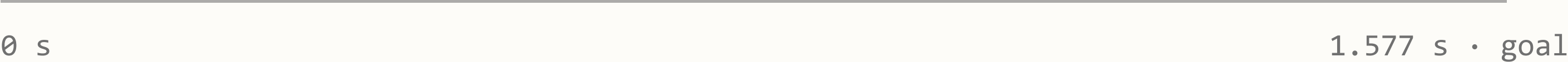
```
PASSED [main.assertion.1]  
      line 7  assertion x == 10  
** 0 of 3 properties failed, 3 passed  
  
VERIFICATION SUCCESSFUL
```

满足两家族确认之后结束目标

本机记录：2026-10-06；下面是目标与 attempt 的 wall_time，非 CLI 端到端

CBMC **SAFE • attempt 1.076 s**

ESBMC **SAFE • attempt 1.419 s**



解析两份已完成结果 → 不同家族数 = 2 → REQUIREMENTS_MET

一票 SAFE 时仍需等待；同批出现相反结果，优先报告 CONFLICT。

返回结论，也返回结论的来源

CLI verify --json → workflow 结果 → goal.report.result / attempts / artifacts

实际 result.json 节选

```
{
  "execution_id": "76b5d09d25784a868368601ab8b0a36a",
  "goal_id": "safe-assert-crosscheck",
  "verdict": "SAFE",
  "confirmations": 2,
  "requirement_satisfied": true,
  "cache_hit": false,
  "status": "COMPLETED",
  "termination_reason": "REQUIREMENTS_MET",
  "wall_time_sec": 1.5767692249974061
}
```

执行目录（前缀节选）

/data/quickstart-.../executions/76b5.../<attempt_id>/

25 个工件引用

逻辑目标 / 物理计划 / 候选得分
命令 / spec / outcome / 容器身份
stdout / stderr / 事件 / 结果
每个工件记录 SHA-256 和来源

命中缓存仍校验证据，仍产生新执行记录

逻辑目标相同 + 环境身份相同 + 工件完整 + 确认要求足够

CacheLookup + fallback

```
CacheLookup(  
  semantic_key="c61c1d87...",  
  min_confirmations=2,  
  fallback=Sequence(  
    Parallel(CBMC, ESBMC), CPAchecker  
  )  
)
```

实际重复结果节选

```
"verdict": "SAFE"  
"confirmations": 2  
"cache_hit": true  
"attempt_ids": []  
"wall_time_sec": 0.01679892100219149  
"source_execution_id":  
  "76b5d09d25784a868368601ab8b0a36a"
```

缓存身份包含工具版本 / 配置 / 精确镜像；复核 SHA-256 与原始来源。

冲突历史使缓存失效；旧工件留存。此例没有启动验证命令，仍可能有工具探测。

工作流边表示执行门槛，不自动组合定理

当前 ready goals 顺序执行；每个 goal 内部可以并行工具

情况	当前输出或执行行为	含义
前置 goal 达到指定 verdict + 确认数	后继 goal 才进入 ready	控制依赖，不传播逻辑假设
超时 / 展开不足 / 不支持 / 不完整 日志	UNKNOWN；必要时后继 BLOCKED	不能当作 SAFE 或具体反例
看到相反确定结果	CONFLICT，优先于多数票	需要保留与调查两方证据
只有一族结论，要求两族	requirement_satisfied=false	不能自动降低确认要求

LLM planner 当前只校验并选择已有形式化目标；无任意分解或自动证明生成。

工具扩展已运行，覆盖范围仍然很小

既有 M11: 128 tests 通过 (112 unit + 16 integration) ; 本次汇报复核原始记录

路径	真实结果	边界
CBMC / ESBMC / CPAchecker	6 个 C 小例: 4 SAFE / 2 UNSAFE 双家族 12 次、三家族 18 次尝试	基础断言路径; 不是大型 benchmark
Ultimate / Eva	Ultimate: 4 SAFE / 2 UNSAFE Eva: 4 SAFE / 2 UNKNOWN	扩展矩阵按 min_confirmations=1 运行
100 次循环	CBMC / ESBMC: UNKNOWN Ultimate ~33.94 s; Eva ~1.50 s: SAFE	固定展开 64; 单案例耗时不可泛化
WP + Z3 / CVC5	两次独立运行均报告 4/4 义务通过	共享 WP 编码; solver_report 有 Skipped RTE guards 警告
独立 Rocq proof_check	正确证明 VERIFIED 错配命题 / Admitted 均 REJECTED	kernel_checked; 不是 C SAFE

成本、预算与复用仍是可研究的原型

已有接口可用；下列限制需要实验数据与新语义来推进

当前做法	实际限制	可以检验的研究问题
确定率 / 中位耗时启发式	粗粒度 tiny 桶；提前取消造成样本偏差	在固定证据门槛下，怎样降低超时率？
等分时间片 / 静态内存份额	无法跨阶段借用资源；ready goals 串行	动态资源借用是否增加完成率？
精确目标缓存	不复用部分证明、VC 或 witness	工件粒度复用怎样保持语义身份？
能力矩阵 + 六个小案例	memory_safety 已解析但无可用 adapter 无法支持广泛性能或健全性结论	扩大语料并验证解析器与结果边界
Local backend + Docker 包装	远程 staging / 环境资格尚未定义	在不同 backend 上如何保持证据一致？

Rocq 检查上层写好的命题与证明

可执行 version=0.2 / proof_check; 上层负责 .v, 运行时负责检查

Expected.v + Correct.v 节选

```
(* Expected.v *)
Definition obligation : Prop :=
  forall (P : Prop), P -> P.

(* Correct.v *)
From VRGoal Require Import Expected.
Lemma discharge : Expected.obligation.
Proof.
  unfold Expected.obligation.
  intros P evidence. exact evidence.
Qed.
```

proof_check DSL 的核心字段

```
"goal": {
  "kind": "proof_check", "namespace": "VRGoal",
  "statement": {"file": "Expected.v",
    "symbol": "VRGoal.Expected.obligation"},
  "proof": {"language": "Rocq",
    "files": ["Correct.v"],
    "symbol": "VRGoal.Correct.discharge"}
},
"requirements": {
  "evidence": "kernel_checked", "axioms": []
}
```

输出 VERIFIED / code_verdict=null; 不增加 C 家族票, 不进入 C 结果缓存。

编译通过之外，还要绑定类型与审计假设

当前闭合策略：拒绝用户 Axiom / Parameter / Admitted 与未核验扩展

依赖排序编译 → 精确类型绑定 → 假设闭包审计 → rocq check 复查

运行时生成 Binding.v; 它只检查已有证明是否匹配

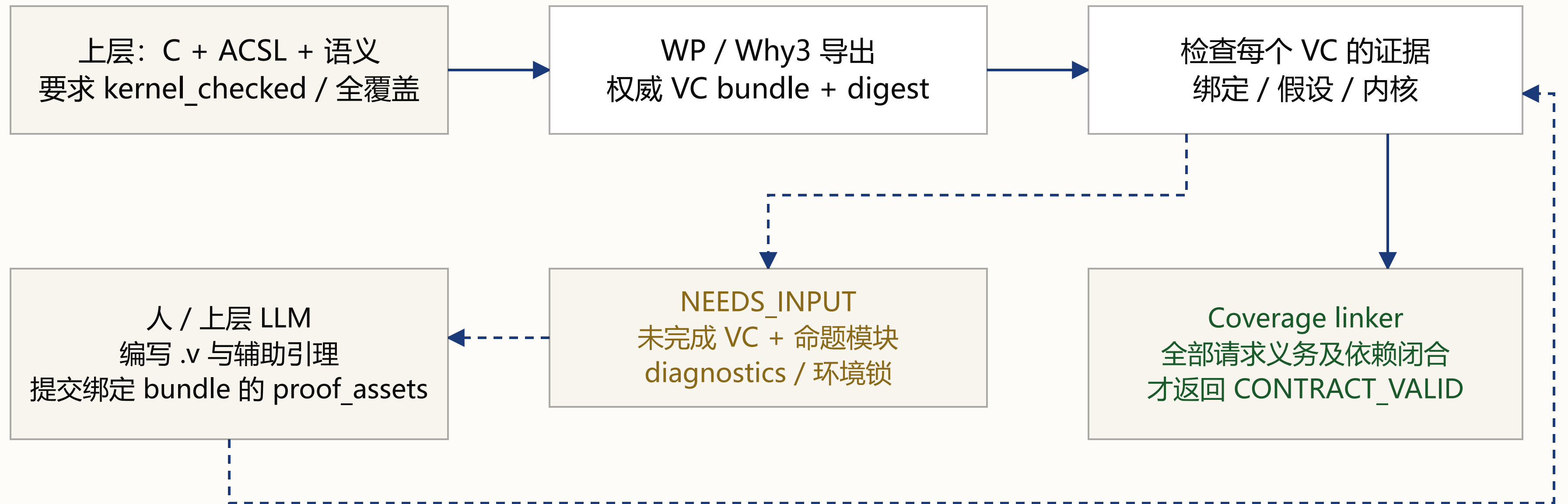
```
Definition required_statement : Prop :=
  VRGoal.Expected.obligation.

Definition bound_proof : required_statement :=
  VRGoal.Correct.discharge.
```

提交内容	真实检查结果	解释
正确证明	VERIFIED	提交证明建立了提交命题
错配命题 / Admitted	REJECTED	proof_type_mismatch / untrusted_axioms_or_extensions

$C \rightarrow VC \rightarrow \text{上层证明} \rightarrow \text{覆盖闭包}$

不能把 Frama-C 与 Rocq 简单串联后当作两票 C SAFE



等待补证时释放容器；上层下一次显式提交。结论仍相对于契约前置条件。

表达证据强度与覆盖，附带命题绑定

version=0.2-proposal / c_contract 是设计稿，不能直接传给当前 verify

第一轮：声明 C 契约目标与所需证据

```
{
  "version": "0.2-proposal",
  "goal": {
    "kind": "c_contract", "program": {...},
    "contract": {"format": "ACSL", ...},
    "semantics": {"integer_arithmetic": "bounded_c",
                  "memory": "iso_c_objects", ...}
  },
  "requirements": {
    "evidence": "kernel_checked",
    "coverage": ["contract", "runtime_errors"],
    "user_axioms": []
  },
  "proof_assets": [], "budget": {...}
}
```

第二轮：补交 .v；结构节选

```
"proof_assets": [{
  "format": "Rocq", "files": ["SquareProof.v"],
  "bundle": {"artifact_id": "<sealed digest>"},
  "bindings": [{
    "obligation_id": "typed_square_ensures",
    "statement_sha256": "<权威命题 digest>",
    "proof_symbol":
      "SquareProof.square_nonnegative"
  }]
}]
```

仍不指定工具或顺序；过期 bundle、命题错配、遗漏新增 VC 均须拒绝。

先验证强证据闭包，再评估调度收益

把研究结论约束在固定命题、固定预算与固定信任要求之内

优先级	可落地实验	验收指标
P1 · 权威 VC 与 coverage	一个函数子集：封存全部 VC / 理论错配、漏证、过期、额外假设负例	覆盖率；错误接受为零 可追溯的 $C \rightarrow VC \rightarrow \text{proof}$ 来源
P2 · 调度与资源	扩大语料；静态 portfolio 作对照 处理超时 / 取消样本；消融成本模型	固定门槛下的完成率 / wall time 超时率 / 峰值资源 / 校准误差
P3 · 工件缓存与后端	VC / proof 粒度复用 显式 staging / 身份 / 清理契约	语义变更失效；后端证据一致性 故障后无泄漏、可恢复

本机已有镜像：一次命令复现双确认与缓存

```
pwsh -NoProfile -File scripts/quickstart.ps1
```